



Zerobus Direct Write API Bug Bash

Author(s)	Rastko Sapić Branko Grbić
Organization	Belgrade ZeroBus Team
Status	Postbash ▾

Preparation

Please read all the steps first to get familiar with all the details, and then move on to executing all of them:

1. Log into [Dogfood workspace](#) and locate or create a table for ingesting. You can use notebook for creating a table

If you don't have a catalog, you can use `zerobus`.

Note: We support only managed tables.

Note: Erase table if exists with: `DROP TABLE IF EXISTS ...`

Example:

Sample table created with Notebook.

```
%sql
```

```
create table zerobus.default.test_table(  
  id INT, name String  
)
```

2. [[Dogfood](#)] Navigate on "Catalog" using left-hand side panel -> navigate to your table -> permissions -> grant -> ALL PRIVILEGES, USE SCHEMA, USE CATALOG

3. Log into [PrPr-enabled Workspace](#).

You should get permissions for this workspace - ask somebody on [#shinkansen-bug-bash](#) Slack channel to enable it

IMPORTANT:

PrPr-enabled workspace is being used only for generating PAT token - don't create tables on it.

4. Issue a [PAT token](#) on *PrPr-enabled workspace*

Go to: Workspace -> Click user located on top right -> Settings -> Developer -> Access Token -> Manage -> Generate New Token

5. Install Python SDK

Run the following:

```
# Download .whl from the link  
> mkdir test_dir  
# Move the .whl to the test_dir  
> cd test_dir  
> python3.9 -m venv venv39  
> source venv39/bin/activate  
> pip install databricks_zerobus-0.0.15-py3-none-any.whl # align to the actual  
name if needed
```

Organizers have built the Python wheel (.whl extension) for you. You just need to install it with python*. You can find the wheel in [#shinkansen-bug-bash](#) Slack channel - download it and place it in some `<test_directory>` (you can place it outside of any repo).

**Requirement: Python 3.9+ (should be preinstalled)*

6. Generate proto with Python wheel

```
> cd test_dir # If not already in the test_dir  
> generate_proto --uc-endpoint  
https://dbc-e2f0eb31-2b0e.staging.cloud.databricks.com --uc-token <PAT_TOKEN>  
--table <catalog.schema.table> --output <proto_name>.proto
```

```
> generate_proto -h # To see all the arguments for the generate_proto command
```

`<proto_name>.proto` that was created should look something like this:

```
syntax = "proto3";  
message TestTable {  
  optional int32 id = 1;  
  optional string name = 2;  
}
```

7. Compile proto-message definition

```
> cd test_dir # Move to test_dir if not already there
```

```
> pip install grpcio-tools
> python3 -m grpc_tools.protoc -I. --python_out=. <proto_name>.proto
```

Python will generate `<proto_name>_pb2.py` file, which you need to import into your Python script for ingesting.

8. Create a Python client for ingesting rows

```
> touch client.py # Create client file
> cursor client.py # Open with your IDE
```

- + Set up the constant variables such as PAT token generated

Example:

Create `<client_filename>.py` file

```
import asyncio
# CHANGE IMPORT TO MATCH YOUR PROTO
import proto_name_pb2
from zerobus_sdk import ZerobusSdk, TableProperties

# CHANGE CONSTANTS TO MATCH YOUR USECASE
DATABRICKS_WORKSPACE_URL = "https://dbc-e2f0eb31-2b0e.staging.cloud.databricks.com"
TABLE_NAME = "zerobus.default.ingest_table"
PAT = "<YOUR_TOKEN>" # IMPORT TOKEN
SERVER_ENDPOINT = "ingest.staging.cloud.databricks.com"

sdk_handle = ZerobusSdk(SERVER_ENDPOINT, DATABRICKS_WORKSPACE_URL, PAT)

async def blocking_call():
    # Create stream to table.
    # CHANGE DESCRIPTOR
    table_properties = TableProperties(TABLE_NAME, proto_name_pb2.TestTable.DESRIPTOR)
    stream = await sdk_handle.create_stream(table_properties)

    # blocking call which completes when the message is durable.
    # INGEST ROWS
    record_future = await stream.ingest_record(
        proto_name_pb2.TestTable(id=1, name="John Doe")
    )

    ack1 = await record_future
    print(f"Record with ID: {ack1} persisted.")

    # Close the stream
    await stream.close()
    print("Stream closed.")

if __name__ == "__main__":
    asyncio.run(blocking_call())
```

Note:

To run this specific file, change import to match your proto, but also change constants, descriptors and `ingest_record()` method to match your desired table.

9. Run!

```
> python3 <client_filename>.py
```



Known issues & pending changes

- ☐ Empty (null) fields to be ingested will be converted to have a default value defined by proto



Test Cases

— Ingesting Rows

The goal of this section is to test if the Python SDK ingests rows into your table. Are the ingested rows correct? Feel free to add new test cases and break the existing ones.

	Test case	Assignee 1	Assignee 2	Comments (LGTM? Bugs? Screenshots?)	Ticket/PR
SP1	100 rows	David M...		LGTM	

— 100 column test

	Test case	Assignee 1	Assignee 2	Comments (LGTM? Bugs? Screenshots?)	Ticket/PR
SP1	3000 INT columns	Filip Ja...		Works	
	100 threads, 1000 iterations of generating 100 columns	David M...		LGTM Code in the Slack thread	

— Concurrency

	Test case	Assignee 1	Assignee 2	Comments (LGTM? Bugs? Screenshots?)	Ticket/PR
SP1		Marko ...		Works (50 concurrent streams)	
		⚙ Person		✔ works like charm <pre>Python import asyncio import concurrent.futures import random # CHANGE IMPORT TO MATCH YOUR PROTO import shinks_test_tbl_pb2 import numpy as np from datetime import datetime from ingest_api_sdk import IngestApiSdk, TableProperties # CHANGE CONSTANTS TO MATCH YOUR USECASE DATABRICKS_WORKSPACE_URL = "https://dbc-e2f0eb31-2b0e.staging.cloud.databricks.com" TABLE_NAME = "main.milan_schema.shinkansen_test_4" PAT = "<>" # IMPORT TOKEN SERVER_ENDPOINT = "ingest.staging.cloud.databricks.com" sdk_handle = IngestApiSdk(SERVER_ENDPOINT, DATABRICKS_WORKSPACE_URL, PAT) async def ingest_records(thread_id, num_records): # Create stream to table. # CHANGE DESCRIPTOR table_properties = TableProperties(TABLE_NAME, shinks_test_tbl_pb2.shinkansen_test_4.DESRIPTOR) stream = await sdk_handle.create_stream(table_properties) now = datetime.now() # Encode date as YYYYMMDD (int32) date_int32 = np.int32(now.year * 10000 + now.month * 100 + now.day) # Encode time as HHMMSSmmm (int64) time_int64 = np.int64(now.hour * 10000000 + now.minute * 100000 + now.second * 1000 + int(now.microsecond / 1000)) # Collect futures for records record_futures = [] for i in range(num_records): record_future = stream.ingest_record(shinks_test_tbl_pb2.shinkansen_test_4(col_INT=i + thread_id * num_records, col_STRING=f"Thread-{thread_id}", col_FLOAT=5.5, col_LONG=2147483648, col_SHORT=32768, col_DOUBLE=5.5, col_BOOLEAN=False, col_BINARY=b'\x48\x65\x6c\x6c\x6f', col_DATE=date_int32, col_TIMESTAMP=time_int64)) record_futures.append(record_future) # Add a random delay between 0 and 5ms await asyncio.sleep(random.uniform(0, 0.005)) # Await all futures acks = await asyncio.gather(*record_futures) for ack in acks: print(f"Thread-{thread_id}: Record with ID: {ack} persisted.") # Close the stream</pre>	

				<pre> await stream.close() print(f"Thread-{thread_id}: Stream closed.") async def main(): num_threads = 16 records_per_threa d = 10000 # Create a thread pool executor with concurrent.futures.ThreadPoolExecutor(max_workers=num_threads) as executor: loop = asyncio.get_event_loop() tasks = [loop.run_in_executor(executor, asyncio.run, ingest_records(thread_id, records_per_thread)) for thread_id in range(num_threads)] await asyncio.gather(*tasks) if __name__ == "__main__": asyncio.run(main())</pre>	
	100 threads, open stream, insert 1000 rows, close stream	David M...		LGTM <pre>Python import asyncio import test_pb2 import threading import time from ingest_api_sdk import IngestApiSdk, TableProperties # CHANGE CONSTANTS TO MATCH YOUR USECASE DATABRICKS_WORKSPACE_URL = "https://dbc-e2f0eb31-2b0e.staging.cloud.databricks.com" TABLE_NAME = "dmilicevic.shinkansen.test_table" PAT = "<>" SERVER_ENDPOINT = "ingest.staging.cloud.databricks.com" sdk_handle = IngestApiSdk(SERVER_ENDPOINT, DATABRICKS_WORKSPACE_URL, PAT) async def nonblocking_call(stream, thread_id, thread_name, counter): record_future = await stream.ingest_record(test_pb2.test_table(id=thread_id, name=thread_name, submit_counter = counter, last_submitted = int(time.time()) * 1000000)) async def thread_code(thread_id, thread_name): # Create stream to table. table_properties = TableProperties(TABLE_NAME, test_pb2.test_table.DESRIPTOR) stream = await sdk_handle.create_stream(table_properties) for i in range(100): await nonblocking_call(stream, thread_id, thread_name, i) # Close the stream await stream.close() def thread_run(thread_id, thread_name): asyncio.run(thread_code(thread_id, thread_name)) if __name__ == "__main__": threads = [] for thread_id in range(100): thread_name = 'Thread ' + str(thread_id) t = threading.Thread(target=thread_run, args=(thread_id, thread_name,)) threads.append(t) for t in threads: t.start() for t in threads: t.join()</pre>	

— Open stream -> ingest -> close stream repeatedly

	Test case	Assignee 1	Assignee 2	Comments(LGTM? Bugs? Screenshots?)	Ticket/PR
SP1		Marko ...		Works(tested ~100 cycles)	

	100 iterations, open stream, insert, close stream	David M...		LGTM	
--	--	------------	--	------	--

— Write to two tables at the same time

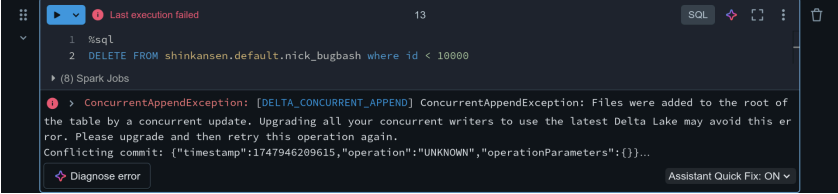
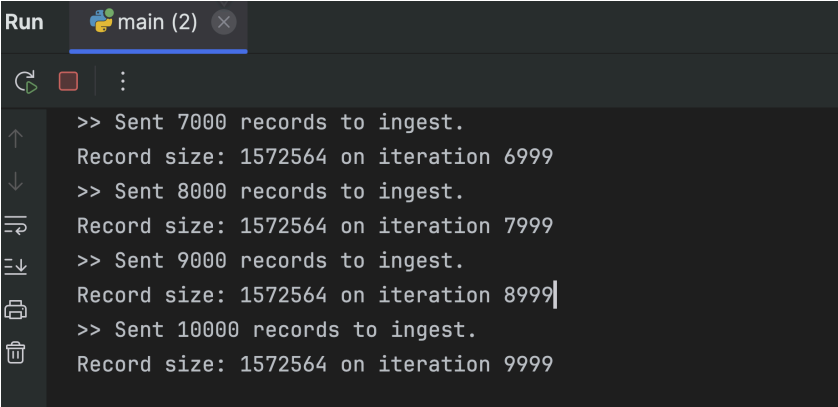
	Test case	Assignee 1	Assignee 2	Comments(LGTM? Bugs? Screenshots?)	Ticket/PR
SP1					

— Other

	Test case	Assignee 1	Assignee 2	Comments(LGTM? Bugs? Screenshots?)	Ticket/PR
SP1	Large strings (1st test - 1000 2nd test - 10k chars) 10k records per thread Random 0-2ms sleep between threads 16 threads	Milan St...		Working perfect  Python <pre>from datetime import datetime, date import random import string import asyncio import concurrent.futures import random # CHANGE IMPORT TO MATCH YOUR PROTO import shinks_test_tbl_pb2 import numpy as np from datetime import datetime from ingest_api_sdk import IngestApiSdk, TableProperties # CHANGE CONSTANTS TO MATCH YOUR USECASE DATABRICKS_WORKSPACE_URL = "https://dbc-e2f0eb31-2b0e.staging.cloud.databricks.com" TABLE_NAME = "main.milan_schema.shinkansen_test_4" PAT = "<>" # IMPORT TOKEN SERVER_ENDPOINT = "ingest.staging.cloud.databricks.com" sdk_handle = IngestApiSdk(SERVER_ENDPOINT, DATABRICKS_WORKSPACE_URL, PAT) async def ingest_records(thread_id, num_records): # Create stream to table. # CHANGE DESCRIPTOR table_properties = TableProperties(TABLE_NAME, shinks_test_tbl_pb2.shinkansen_test_4.DESRIPTOR) stream = await sdk_handle.create_stream(table_properties) # Get the current time now = datetime.now() # Get ticks as the number of microseconds since the epoch ticks = int(now.timestamp() * 1e6) now = datetime.now() # Get today's date today = date.today() # Convert to integer in YYYYMMDD format date_as_int = today.year * 10000 + today.month * 100 + today.day random_string = ''.join(random.choices(string.ascii_letters + string.digits, k=1000)) # Collect futures for records record_futures = [] for i in range(num_records): record_future = await stream.ingest_record(shinks_test_tbl_pb2.shinkansen_test_4(col_INT=i + thread_id * num_records, col_STRING=random_string, col_FLOAT=5.5, col_LONG=2147483648, col_SHORT=32768, col_DOUBLE=5.5, col_BOOLEAN=False, col_BINARY=b'\x48\x65\x6c\x6c\x6f', col_DATE=date_as_int, col_TIMESTAMP=ticks)) record_futures.append(record_future) # Add a random delay between 0 and 5ms await asyncio.sleep(random.uniform(0, 0.005)) # Await all futures acks = await asyncio.gather(*record_futures) for ack in acks: print(f"Thread-{thread_id}: Record with ID: {ack} persisted.")</pre>	

				<pre># Close the stream await stream.close() print(f"Thread-{thread_id}: Stream closed.") async def main(): num_threads = 16 records_per_thread = 10000 # Create a thread pool executor with concurrent.futures.ThreadPoolExecutor(max_workers=num_threads) as executor: loop = asyncio.get_event_loop() tasks = [loop.run_in_executor(executor, asyncio.run, ingest_records(thread_id, records_per_thread)) for thread_id in range(num_threads)] await asyncio.gather(*tasks) if __name__ == "__main__": asyncio.run(main())</pre>	
--	--	--	--	--	--

 Follow-up

Status	JIRA/PR	Description	Issue type	Priority	Owner	GA blocker
Wont ...	Nick	Error deleting while streaming data in: 	By design This is expected. When there is a running ingest stream, if someone tries to delete data from target table then there will be a conflict and one operation needs to fail. This is by design.			No
Todo	Vicky	1.57MB messages, after 10,000 rows of ingestion the Python client stalls. It seems to get stuck at the point of which the internal queue is full.  Shinkansen - Row Ingest API Bug Bash	Bug https://github.com/databricks-eng/universe/issues/1123942	Vicky to send code. Someone to check. Big messages case.		Yes
Done	Nick	How to convert to int32/int64 for date types. Onboarding says to use epoch microseconds, but that won't fit in a 32 bit int. My guess is you want to do what parquet does, and you actually mean to convert DATE to "days since the epoch" and TIMESTAMP to "microseconds since the epoch" <- This works in that if you do this a notebook will see the correct files	Bug	TODO - follow up with Nick and update the doc. Updated documentation: Date Should be converted to int32 (epoch time in seconds). Timestamp Should be converted to int64 (epoch time in microseconds)		Yes
Todo	Vicky	Got a huge stacktrace which is hard to parse when ingesting large	Improvement	Vicky to send code to		Yes

		messages for an extended period of time on a single stream (recovery was turned off). We should clean this up. → Stacktrace	https://github.com/databricks-eng/universe/issues/1123831	Vedran to repro. Investigate why we have such long stacktrace		
Todo ▾		Vladimir Golubev : Insert into a collated table throws Failed to open table for write: CREATE TABLE IF NOT EXISTS dst_collated1(id INT, collated STRING COLLATE UTF8_LCASE); Same thing for generated columns: CREATE TABLE IF NOT EXISTS generated1(id INT, gen STRING GENERATED ALWAYS AS (CAST(id AS STRING))); 2025-05-23T10:30:56.378282Z WARN universe_shinkansen_core_server::grpc_ingest_server::ephemeral_stream_ha ndling: Error returned: Failed to open table for write. Failed to create new transaction for table at path: UC { storage_location: "s3://us-west-2-extstaging-managed-catalog-test-bucket-1/19a85dee-54bc-4 3a2-87ab-023d0ec16013/tables/a302cf4a-0fa3-49f8-89b8-ac9c212378bb", name: "main.vladimir_golubev_shinkansen_bugbash.dst_collated1", is_managed_table: true }. Error: Unsupported: Unknown WriterFeatures: "domainMetadata", "collations-preview". Supported WriterFeatures: "appendOnly", "deletionVectors", "invariants".	Bug ▾ https://github.com/databricks-eng/universe/issues/1123827	TODO - what can we do with collated tables. Define Colations are defined as delta features. Not much we can do other than error message. We should document that this is not supported. There are also constraints. We need metadata operation from Delta Kernel whether we can write into a target table or not		Yes ▾
Todo ▾		Vladimir Golubev : Creating stream for a view gives a confusing error: CREATE OR REPLACE VIEW v1 AS SELECT 1 AS id, 'abc' AS name; <pre>python3 client.py [ERROR][ingest_api_sdk]: Create stream error: <RpcError of RPC that terminated with: status = StatusCode.INTERNAL details = "Unity Catalog error (Error code 54)." debug_error_string = "UNKNOWN:Error received from peer ipv4:44.214.111.163:443 (created_time:"2025-05-23T10:57:55.142376683+00:00", grpc_status:13, grpc_m essage:"Unity Catalog error (Error code 54).")"></pre>	Bug ▾ https://github.com/databricks-eng/universe/issues/1123779	TODO - better error message when VIEW is given as destination. For Any object in unity catalog other than managed table return a proper error message.		Yes ▾
Wont ... ▾		Milan Stefanovic : DNS resolution failure when spamming with 100 threads. Ruzel Ibragimov : GRPC uses C-Ares to resolve endpoints: it looks like it gets overheated. Probably it doesn't share DNS caches inside? Milan Stefanovic : [ERROR][ingest_api_sdk]: Create stream error: <AioRpcError of RPC that terminated with: status = StatusCode.UNAVAILABLE details = "DNS resolution failed for ingest.staging.cloud.databricks.com: C-ares status is not ARES_SUCCESS qtype=A name=ingest.staging.cloud.databricks.com is_balancer=0: Could not contact DNS servers" debug_error_string = "UNKNOWN:Error received from peer {created_time:"2025-05-23T10:44:43.020742+00:00", grpc_status:14, grpc_message:"DNS resolution failed for ingest.staging.cloud.databricks.com: C-ares status is not ARES_SUCCESS qtype=A name=ingest.staging.cloud.databricks.com is_balancer=0: Could not contact DNS servers"}" > Python import asyncio import concurrent.futures import random # CHANGE IMPORT TO MATCH YOUR PROTO import shinks_test_tbl_pb2 import numpy as np from datetime import datetime from ingest_api_sdk import IngestApiSdk, TableProperties # CHANGE CONSTANTS TO MATCH YOUR USECASE DATABRICKS_WORKSPACE_URL = "https://dbc-e2f0eb31-2b0e.staging.cloud.databricks.com" TABLE_NAME = "main.milan_schema.shinkansen_test_4" PAT = "dapi6f1ab3a0c879ef2273b32657f0771095" # IMPORT TOKEN SERVER_ENDPOINT = "ingest.staging.cloud.databricks.com" sdk_handle = IngestApiSdk(SERVER_ENDPOINT, DATABRICKS_WORKSPACE_URL, PAT)	By design ▾ This is not Shinkansen issue. This is problem on client side when DNS resolution does not work properly for large number of client threads.	This was from the same process. Single machine issue. This is GRPC implementation and this is problem happening on client side, not related to Shinkansen.		No ▾

		<pre>async def ingest_records(thread_id, num_records): # Create stream to table. # CHANGE DESCRIPTOR table_properties = TableProperties(TABLE_NAME, shinks_test_tbl_pb2.shinkansen_test_4.DESRIPTOR) stream = await sdk_handle.create_stream(table_properties) now = datetime.now() # Encode date as YYYYMMDD (int32) date_int32 = np.int32(now.year * 10000 + now.month * 100 + now.day) # Encode time as HHMMSSmmm (int64) time_int64 = np.int64(now.hour * 10000000 + now.minute * 100000 + now.second * 1000 + int(now.microsecond / 1000)) # Collect futures for records record_futures = [] for i in range(num_records): record_future = stream.ingest_record(shinks_test_tbl_pb2.shinkansen_test_4(col_INT=i + thread_id * num_records, col_STRING=f"1-Thread-{thread_id}", col_FLOAT=5.5, col_LONG=2147483648, col_SHORT=32768, col_DOUBLE=5.5, col_BOOLEAN=False, col_BINARY=b'\x48\x65\x6c\x6c\x6f', col_DATE=date_int32, col_TIMESTAMP=time_int64)) record_futures.append(record_future) # Add a random delay between 0 and 5ms await asyncio.sleep(random.uniform(0, 0.005)) # Await all futures acks = await asyncio.gather(*record_futures) for ack in acks: print(f"Thread-{thread_id}: Record with ID: {ack} persisted.") # Close the stream await stream.close() print(f"Thread-{thread_id}: Stream closed.") async def main(): num_threads = 100 records_per_thread = 10000 # Create a thread pool executor with concurrent.futures.ThreadPoolExecutor(max_workers=num_threa ds) as executor: loop = asyncio.get_event_loop() tasks = [loop.run_in_executor(executor, asyncio.run, ingest_records(thread_id, records_per_thread)) for thread_id in range(num_threads)] await asyncio.gather(*tasks) if __name__ == "__main__": asyncio.run(main())</pre>			
Todo ▾		A lot of errors from async IO, when client is not used properly.	Bug ▾ https://github.com/databricks-eng/universe/issues/1123838	Just write a proper example for clients how to achieve parallelism. Write one pattern and one antipattern.	Yes ▾

Appendix: Extra content

PrPr onboarding doc: go.RLI/prpr-onboarding